LAB 21.1 AIAC

2303A51494

B27

Task 1 – AI-Assisted Pair Programming

PROMPT:

generate an initial program eg a simple to do list manager and suggest enhancements such as error handling , adding features or refactoring compare the original and improved version

CODE:

```
#generate an initial program eg a simple to do list manager and suggest
#enhancements such as error handling , adding features or refactoring compare the original and improved version
# Original version
to_do_list = []
Windsurf: Refactor | Explain | Generate Docstring | ×
def add_task(task):
    to_do_list.append(task)
Windsurf: Refactor | Explain | Generate Docstring | ×
def view_tasks():
    for index, task in enumerate(to_do_list, start=1):
        print(f"{index}. {task}")
# Example usage
add_task("Buy groceries")
add_task("Complete assignment")
print("To-Do List:")
view_tasks()
```

```python
# Improved version with error handling and additional features
to_do_list = []
Windsurf: Refactor | Explain | Generate Docstring | ×
def add_task(task):
    if not task:
        print("Error: Task cannot be empty.")
        return
    to_do_list.append(task)
    print(f"Task '{task}' added successfully.")
Windsurf: Refactor | Explain | Generate Docstring | ×
def view_tasks():
    if not to_do_list:
        print("No tasks in the to-do list.")
        return
    print("To-Do List:")
    for index, task in enumerate(to_do_list, start=1):
        print(f"{index}. {task}")
Windsurf: Refactor | Explain | Generate Docstring | ×
def remove_task(task_number):
    if task_number < 1 or task_number > len(to_do_list):
        print("Error: Invalid task number.")
        return
    removed_task = to_do_list.pop(task_number - 1)
    print(f"Task '{removed_task}' removed successfully.")
# Example usage
add_task("Buy groceries")
add_task("Complete assignment")
```

```python
Windsurf: Refactor | Explain | Generate Docstring | ×
def remove_task(task_number):
    if task_number < 1 or task_number > len(to_do_list):
        print("Error: Invalid task number.")
        return
    removed_task = to_do_list.pop(task_number - 1)
    print(f"Task '{removed_task}' removed successfully.")
# Example usage
add_task("Buy groceries")
add_task("Complete assignment")
add_task("")  # Testing error handling for empty task
print("\nViewing tasks:")
view_tasks()
remove_task(2)  # Removing a task
print("\nViewing tasks after removal:")
view_tasks()
add_task("Go for a walk")
print("\nViewing tasks after adding a task:")
view_tasks()
```

OUTPUT:

```
ents/AI Assisted coding/LAB21.py"
To-Do List:
1. Buy groceries
2. Complete assignment
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppDat
ents/AI Assisted coding/LAB21.py"
Task 'Buy groceries' added successfully.
Task 'Complete assignment' added successfully.
Error: Task cannot be empty.

Viewing tasks:
To-Do List:
1. Buy groceries
2. Complete assignment
Task 'Complete assignment' removed successfully.

Viewing tasks after removal:
To-Do List:
1. Buy groceries
Task 'Go for a walk' added successfully.

Viewing tasks after adding a task:
To-Do List:
1. Buy groceries
2. Go for a walk
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

EXPLANATION:

- The original program allows users to add and view tasks but lacks validation and additional functionality.
- The improved version adds error handling, such as preventing empty tasks and checking valid task numbers.
- It introduces new features like removing tasks and displaying messages for user actions.
- Overall, the improved version is more user-friendly, reliable, and better structured compared to the basic version.

Task 2 – Version Control Integration

PROMPT

Implement version control for a above program to-do list project using Git. Initialize a Git repository and add the initial version of the program. Create a new branch to apply AI-assisted improvements such as error handling and additional features. Commit the changes with meaningful messages.Merge the updated branch back into the main branch. Finally, analyze how version control helps in managing changes, testing improvements safely, and supporting collaborative development.

CODE

```
#Implement version control for a above program to-do list project using Git. Initialize a Git repository and
# add the initial version of the program. Create a new branch to apply AI-assisted improvements such as
# error handling and additional features. Commit the changes with meaningful messages.
# Merge the updated branch back into the main branch. Finally, analyze how version
# control helps in managing changes, testing improvements safely, and supporting collaborative development.
# To implement version control for the to-do list project using Git, follow these steps:
#  Initialize a Git repository:
#git init
#  Add the initial version of the program:
#git add .
#git commit -m "Initial commit - to do list program"
#git checkout -b ai-update explains the process of creating a new branch
# Apply AI-assisted improvements such as error handling and additional features in the new branch.
# Commit the changes with meaningful messages:
#git add . explains the process of staging the changes for commit
# git checkout master explains the process of merging the updated branch back into the main branch
# git merge ai-update explains the process of merging the changes from the ai-update branch into the master branch
# git init
# git add .
# git commit -m "Initial commit - to do list program"
# git checkout -b ai-update
# # (Make changes to the code for error handling and additional features)
# git add .
# git checkout master
# git merge ai-update
```

OUTPUT

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git init
Initialized empty Git repository in C:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/.git/
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git add .
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git commit -m "Initial commit - to do list program"
[master (root-commit) 02934dd] Initial commit - to do list program
 1 file changed, 47 insertions(+)
 create mode 100644 LAB21.py
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git checkout -b ai-update
Switched to a new branch 'ai-update'
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git add .
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git checkout master
Switched to branch 'master'
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git merge ai-update
Already up to date.
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

EXPLANATION

- The task uses Git to manage different versions of the to-do list project.
- A new branch is created to safely add AI-assisted improvements without affecting the original code.
- Changes are committed with clear messages to track what was modified.
- Finally, the updated branch is merged into the main branch, showing how version control helps in organized and collaborative development.

Task 3 – AI for Commit Message Generation

PROMPT: Generate clear and concise Git commit messages for the improvements made in my to-do list project,including validation, error handling, and new features

OUTPUT

```
#Generate clear and concise Git commit messages for the improvements made in my to-do list project,
# including validation, error handling, and new features
1. "Add validation for empty tasks in add task function"
2. "Implement error handling for invalid task numbers in remove task function"
3. "Add feature to view tasks with proper formatting"
4. "Refactor code for better readability and maintainability"
5. "Update documentation to reflect new features and error handling"
#human written vs AI generated commit messages:
# Human-written commit messages often provide a clear and concise description of the changes made,
# while AI-generated commit messages may sometimes be more generic or less specific.
# Human-written messages may also include context or reasoning behind the changes,
# whereas AI-generated messages may focus solely on the technical aspects of the changes.
# Overall, human-written commit messages tend to be more personalized and tailored to the specific project,
# while AI-generated messages may lack the nuance and understanding of the project's goals and requirements.
```

EXPLANATION

- The task uses AI to automatically generate clear and meaningful Git commit messages based on code changes in the to-do list project.
- These AI-generated messages are compared with human-written ones to check clarity, consistency, and usefulness.
- AI messages are usually more structured and descriptive, while human messages may be shorter or unclear.
- This helps improve documentation and traceability, making it easier to understand project changes over time

Task 4 – Pair Programming Simulation

PROMPT

Create a project where one student writes the base code and another improves it using AI, and use Git branching and merging to combine their work.

CODE

```python
#Create a project where one student writes the base code and another improves it using AI,
# and use Git branching and merging to combine their work.
# Student 1 - Base code
to_do_list = []
Windsurf: Refactor | Explain | Generate Docstring | ×
def add_task(task):
    to_do_list.append(task)
Windsurf: Refactor | Explain | Generate Docstring | ×
def view_tasks():
    for index, task in enumerate(to_do_list, start=1):
        print(f"{index}. {task}")
```

```python
# Student 2 - AI improvements
Windsurf: Refactor | Explain | Generate Docstring | ×
def add_task(task):
    if not task:
        print("Error: Task cannot be empty.")
        return
    to_do_list.append(task)
    print(f"Task '{task}' added successfully.")
Windsurf: Refactor | Explain | Generate Docstring | ×
def view_tasks():
    if not to_do_list:
        print("No tasks in the to-do list.")
        return
    print("To-Do List:")
    for index, task in enumerate(to_do_list, start=1):
        print(f"{index}. {task}")
Windsurf: Refactor | Explain | Generate Docstring | ×
def remove_task(task_number):
    if task_number < 1 or task_number > len(to_do_list):
        print("Error: Invalid task number.")
        return
    removed_task = to_do_list.pop(task_number - 1)
    print(f"Task '{removed_task}' removed successfully.")
```

OUTPUT

```
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.p
To-Do List:
1. Buy groceries
Task 'Go for a walk' added successfully.

Viewing tasks after adding a task:
To-Do List:
1. Buy groceries
2. Go for a walk
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> & C:/Users/Pav
```

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> & C:/Users/Pavani/AppData/Local/Micros
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
To-Do List:
1. Buy groceries
2. Complete assignment
Task 'Buy groceries' added successfully.
Task 'Complete assignment' added successfully.
Error: Task cannot be empty.

Viewing tasks:
To-Do List:
1. Buy groceries
2. Complete assignment
Task 'Complete assignment' removed successfully.

Viewing tasks after removal:
To-Do List:
1. Buy groceries
Task 'Go for a walk' added successfully.

Viewing tasks after adding a task:
To-Do List:
1. Buy groceries
2. Go for a walk
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

```
2. Go for a walk
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> & C:/Users/Pavani/AppData/Local/Microsoft/Wind
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git checkout -b base-code
Switched to a new branch 'base-code'
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git add .
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git commit -m "Add base code for to-do list
>> ^C
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git commit -m "Add base code for to-do list"
[base-code 07feb1f] Add base code for to-do list
 1 file changed, 114 insertions(+), 30 deletions(-)
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git checkout -b ai-improvements
Switched to a new branch 'ai-improvements'
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git add .
```

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git add .
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> git commit -m "Merge base code and AI improvements"
[master 271bf09] Merge base code and AI improvements
 1 file changed, 114 insertions(+), 30 deletions(-)
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

EXPLANATION

- The task demonstrates collaboration where Student A creates the basic to-do list program and Student B improves it using AI-assisted features.
- Git branching is used so both students can work independently without affecting each other's code.
- Each set of changes is committed with meaningful messages to track progress.

- Finally, the branches are merged into the main branch, combining both contributions into a single improved program.

Task 5 – AI-Assisted Debugging

PROMPT

Create a program with some logical or syntax errors. Use AI to identify and correct the errors. Compare the original buggy code with the fixed version and explain the improvements.

CODE

```
# Original buggy code
Windsurf: Refactor | Explain | Generate Docstring | X
def calculate_area(radius):
    area = 3.14 * radius ** 2
    return area
# AI identifies and corrects the error
Windsurf: Refactor | Explain | Generate Docstring | X
def calculate_area(radius):
    if radius < 0:
        print("Error: Radius cannot be negative.")
        return None
    area = 3.14 * radius ** 2
    return area
```

OUTPUT

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> & C:/Users/Pavani/AppData/Local/Micros
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
78.5
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

```
78.5
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1> & C:/Users/Pavani/AppData/Local/Mi
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
Error: Radius cannot be negative.
None
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

EXPLANATION

- The original program calculates the area of a circle but does not handle invalid input like a negative radius.
- This leads to a logical error, as a circle cannot have a negative radius.
- AI improves the code by adding a validation check to prevent negative values and display an error message.
- The corrected version is more reliable and ensures only valid inputs are processed.

Task 6 – Feature Expansion using AI

PROMPT

Create a basic calculator program. Use AI to add features like input validation and history. Explain the improvements.

CODE

```
#Create a basic calculator program.
# Use AI to add features like input validation and history. Explain the improvements.
# Basic calculator program
Windsurf: Refactor | Explain | Generate Docstring | ×
def add(a, b):
    return a + b
Windsurf: Refactor | Explain | Generate Docstring | ×
def subtract(a, b):
    return a - b
Windsurf: Refactor | Explain | Generate Docstring | ×
def multiply(a, b):
    return a * b
Windsurf: Refactor | Explain | Generate Docstring | ×
def divide(a, b):
    if b == 0:
        print("Error: Cannot divide by zero.")
        return None
    return a / b
print(add(5, 3))  # Output: 8
print(subtract(10, 4))  # Output: 6
print(multiply(2, 6))  # Output: 12
print(divide(8, 0))  # Output: Error message for division by zero
```

```python
# AI improvements with input validation and history
history = []
Windsurf: Refactor | Explain | Generate Docstring | ×
def add(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        print("Error: Both inputs must be numbers.")
        return None
    result = a + b
    history.append(f"{a} + {b} = {result}")
    return result
Windsurf: Refactor | Explain | Generate Docstring | ×
def subtract(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        print("Error: Both inputs must be numbers.")
        return None
    result = a - b
    history.append(f"{a} - {b} = {result}")
    return result
Windsurf: Refactor | Explain | Generate Docstring | ×
def multiply(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        print("Error: Both inputs must be numbers.")
        return None
    result = a * b
    history.append(f"{a} * {b} = {result}")
    return result
Windsurf: Refactor | Explain | Generate Docstring | ×
def divide(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        print("Error: Both inputs must be numbers.")
        return None
    if b == 0:
        print("Error: Cannot divide by zero.")
        return None
    result = a / b
    history.append(f"{a} / {b} = {result}")
    return result
```

```python
def multiply(a, b):
    result = a * b
    history.append(f"{a} * {b} = {result}")
    return result
```

Windsurf: Refactor | Explain | Generate Docstring | ×

```python
def divide(a, b):
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        print("Error: Both inputs must be numbers.")
        return None
    if b == 0:
        print("Error: Cannot divide by zero.")
        return None
    result = a / b
    history.append(f"{a} / {b} = {result}")
    return result
print(add(5, 3))  # Valid input
print(subtract(10, 4))  # Valid input
print(multiply(2, 6))  # Valid input
print(divide(8, 0))  # Invalid input (division by zero)
print(divide(8, 2))  # Valid input
```

OUTPUT

```
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
8
6
12
Error: Cannot divide by zero.
None
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

```
n3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/A21.1/LAB21.py"
8
6
12
Error: Cannot divide by zero.
None
4.0
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding\A21.1>
```

EXPLANATION

- The basic calculator performs simple operations like addition, subtraction, multiplication, and division but has limited error handling.
- The AI-improved version adds input validation, ensuring only numbers are accepted and preventing invalid inputs.

- It also introduces a history feature, which stores and displays past calculations for better usability.
- Overall the improved version is more reliable, user-friendly, and better at handling errors compared to the basic version.